

Practical 8(Newton Interpolation)

P- 1 Computing Divided Difference:

```
In[*]:= NthDividedDiff[x0_, f0_, startindex_, endindex_] :=  
Module[{x = x0, f = f0, i = startindex, j = endindex, answer},  
If[i == j, Return[f[[i]], answer =  
(NthDividedDiff[x, f, i + 1, j] - NthDividedDiff[x, f, i, j - 1]) / (x[[j]] - x[[i])];  
Return[answer]]];];
```

Question 1 :

```
In[*]:= x = {0, 1, 3};  
f = {1, 3, 55};  
NthDividedDiff[x, f, 1, 2]
```

```
Out[*]=  
2
```

```
In[*]:= x = {0, 1, 3};  
f = {1, 3, 55};  
NthDividedDiff[x, f, 2, 3]
```

```
Out[*]=  
26
```

```
In[*]:= NthDividedDiff[x, f, 1, 3]  
Out[*]=  
8
```

Question 2:

```
In[*]:= x = {-1, 0, 1, 2};  
f = {5, 1, 1, 11};  
NthDividedDiff[x, f, 1, 2]
```

```
Out[*]=  
-4
```

```
In[*]:= NthDividedDiff[x, f, 2, 3]  
Out[*]=  
0
```

```
In[*]:= NthDividedDiff[x, f, 1, 3]
```

```
Out[*]=  
2
```

```
In[*]:= NthDividedDiff[x, f, 2, 4]
```

```
Out[*]=  
5
```

```
In[*]:= NthDividedDiff[x, f, 1, 4]
```

```
Out[*]=  
1
```

```
In[*]:= NthDividedDiff[x, f, 3, 4]
```

```
Out[*]=  
10
```

P-2:

```
In[*]:= NewtonDDPoly[x0_, f0_] :=  
Module[{x1 = x0, f = f0, n, newtonPolynomial, k, j}, n = Length[x1];  
newtonPolynomial[y_] = 0;  
For[i = 1, i ≤ n, i++, prod[y_] = 1;  
For[k = 1, k ≤ i - 1, k++, prod[y_] = prod[y] * (y - x1[[k])]];  
newtonPolynomial[y_] =  
newtonPolynomial[y] + NthDividedDiff[x1, f, 1, i] * prod[y];  
Return[newtonPolynomial[y]]];
```

```
In[*]:= nodes = {0, 1, 3};  
values = {1, 3, 55};  
NewtonDDPoly[nodes, values]
```

```
Out[*]=  
1 + 2 y + 8 (-1 + y) y
```

Question 2:

```
In[*]:= nodes = {-1, 0, 1, 2};  
values = {5, 1, 1, 11};  
NewtonDDPoly[nodes, values]
```

```
Out[*]=  
5 - 4 (1 + y) + 2 y (1 + y) + (-1 + y) y (1 + y)
```

```
In[*]:= Simplify[5 - 4 (1 + y) + 2 y (1 + y) + (-1 + y) y (1 + y)]
```

```
Out[*]=  
1 - 3 y + 2 y^2 + y^3
```